Simulated E-Banking System
Team #3
Dylan Chamberlin, Regan Cunningham, Jacob Dell, Aiden Grimsey

Software Design Document

Version: 1 **Date: 7 Apr 2026**

TABLE OF CONTENTS

1. INTRODUCTION

1.1 Purpose

The purpose of this Software Design Document is to describe the architecture and the detailed system design on the Simulated E-Banking System (SEBS). It is intended to serve as the main reference for the development of the SEBS during implementation, and to communicate the design choices to all stakeholders.

1.2 Scope

The SEBS is a locally hosted, web-based application that simulates the functionality of an online banking platform using mock data and simulated transactions. The system allows the user to log in, view their accounts, check balances, transfer funds to another account, and review the history of their transactions. It also provides an administrative interface for activity monitoring and managing accounts. The SEBS does not connect or use any real financial data or banking institutions.

1.3 Overview

Section 2 will provide a general system overview. Section 3 will describe the architecture of the system and the rationale behind the design choices. Section 4 will define the way the data is designed Section 5 details each component of that design. Section 6 will cover the interface design seen by users. Section 7 will show a requirements traceability matrix

1.4 Reference Material

- IEEE Std 1016-2009 — IEEE Standard for Information Technology: Systems Design — Software Design Descriptions
- IEEE Std 1058-1998 — IEEE Standard for Software Project Management Plans (cited in your SPMP)
- Simulated E-Banking System Software Project Management Plan (SPMP)
- Simulated E-Banking System Software Requirements Specification (SRS)

1.5 Definitions and Acronyms

SEBS - Simulated E-Banking System
SRS - Software Requirements Specification
SDD - Software Design Document
MVT - Model-View-Template

2. System Overview

SEBS is a web-based simulated online banking platform built for educational and demonstration purposes. It is locally hosted and accessible through any standard web browser. It supports two different user roles. One is the regular user, who can authenticate, view their account balance, transfer funds to other accounts and view their transaction history. Administrators can monitor the general system activity and manage any test accounts that are created.

The system is built using Python and Django for the backend. HTML is used for the frontend and a local database such as SQLite is used for data storage. Version control is managed and kept track of via Github.

SEBS is not connected to any real banking network, payment system, or financial institution. All accounts and transactions are simulated.

3. System Architecture

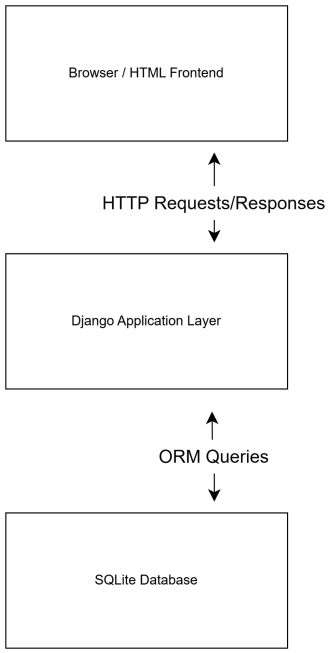
3.1 Architectural Design

The presentation tier will involve HTML pages rendered in the user's browser. They will handle all user interaction and display information returned from the application layer/. The application tier is the Django backend, which contains all the business logic. Authentication, transaction processing, account management, and admin functionality will take place in Django's Model-View-Template (MVT) pattern. The data tier involves the database that is managed through Django's ORM. It stores all the simulated users, accounts, and transactions. The external actors are as follows:

Regular User - interacts with login, dashboard, transfer, and history pages.

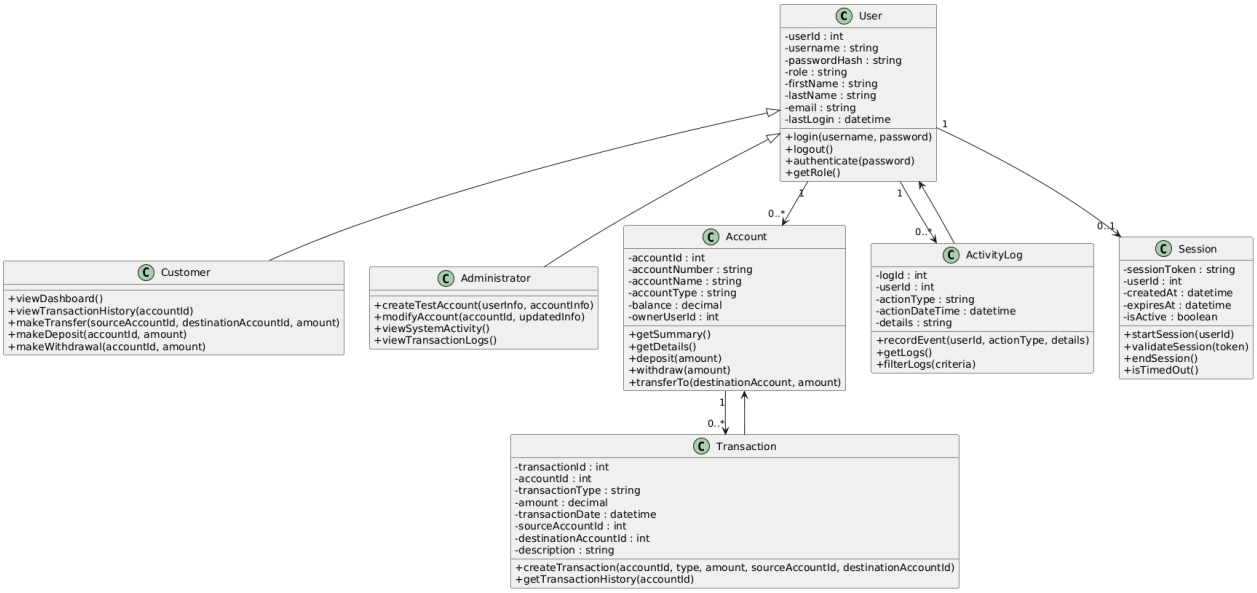
Administrator - interacts with the admin panel.

Web Browser - the way both regular users and administrators will access the system.

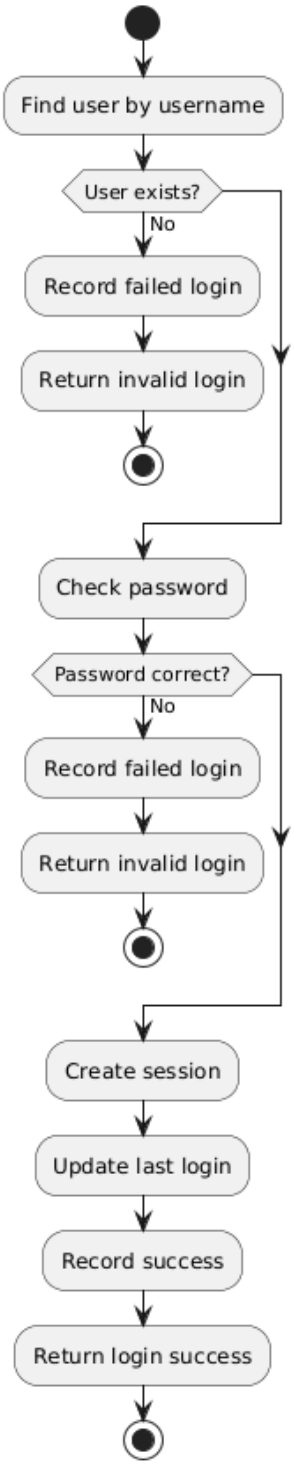

Browser / HTML Frontend
HTTP Requests/Responses
Django Application Layer
ORM Queries
SQLite Database

3.2 Decomposition Description

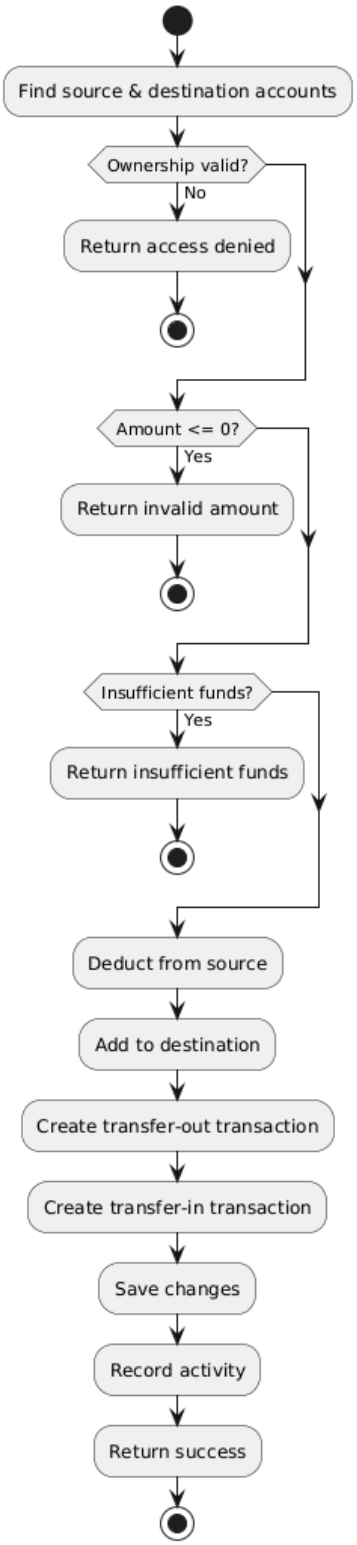
Class Diagram



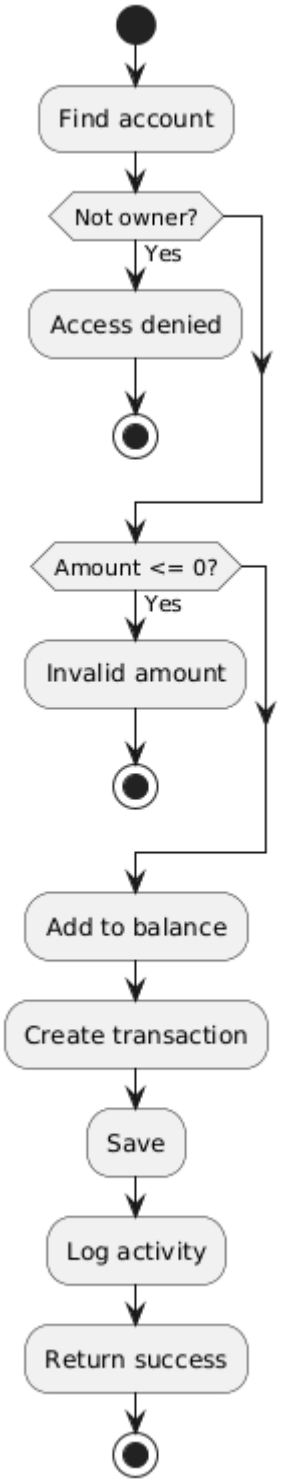
User Login


Find user by username
User exists?
No
Record failed login
Return invalid login
Check password
Password correct?
No
Record failed login
Return invalid login
Create session
Update last login
Record success
Return login success

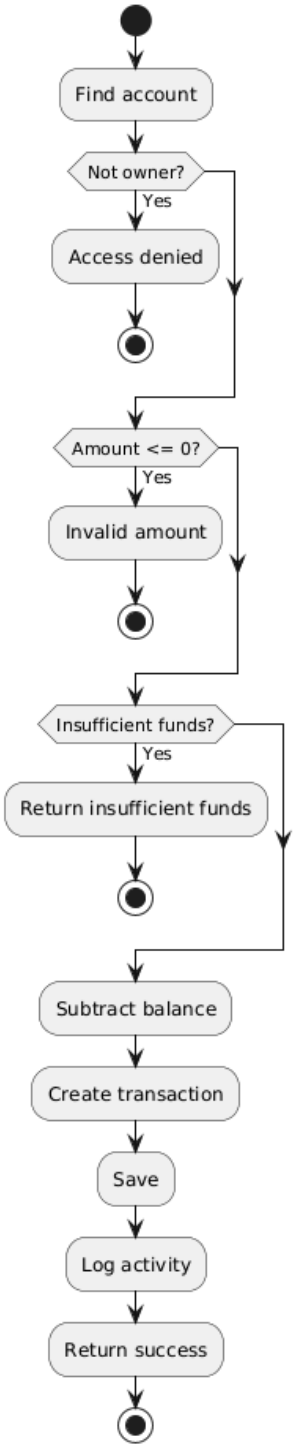
Customer Transfer



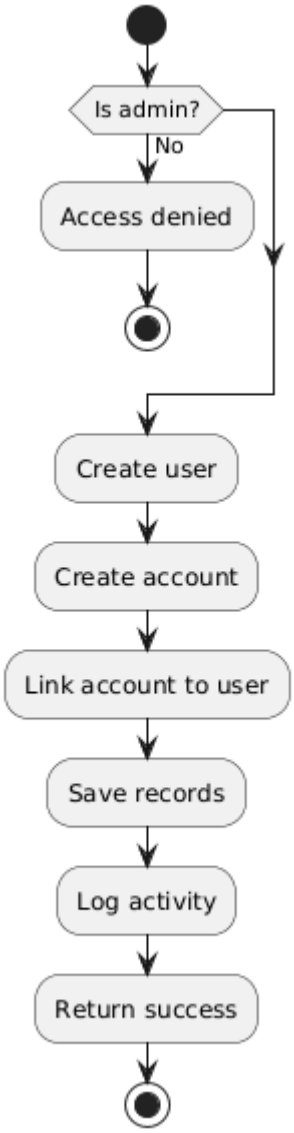
Deposit


Find account
Not owner?
Yes
Access denied
Amount <= 0?
Yes
Invalid amount
Add to balance
Create transaction
Save
Log activity
Return success

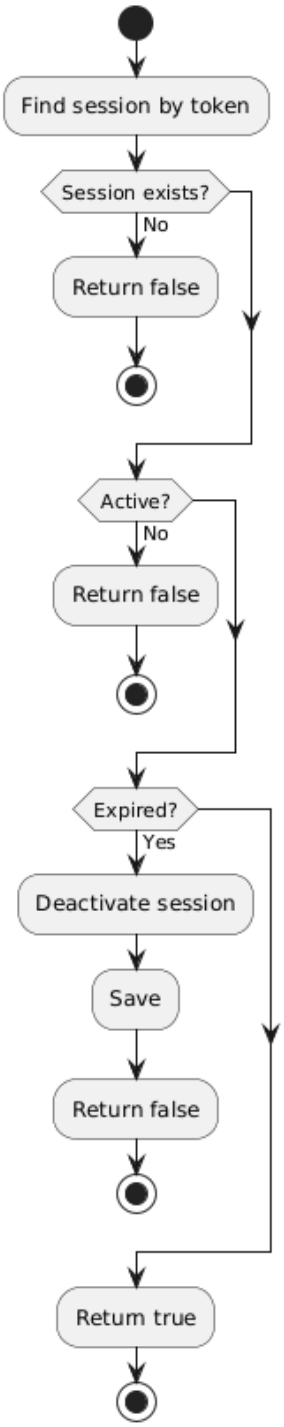
Withdrawal


Find account
Not owner?
Yes
Access denied
Amount <= 0?
Yes
Invalid amount
Insufficient funds?
Yes
Return insufficient funds
Subtract balance
Create transaction
Save
Log activity
Return success

Admin: Create Account



Session validation



3.3 Design Rationale

Django was chosen because of its built-in MVT structure that cleanly separates the data, logic, and presentation. It also includes built-in CSRF protection, session management, and an ORM, which are all important when simulating something like a banking application. All together, these features reduce implementation time and security risks.

A 3-tier architecture was chosen because it is the standard model for web applications and mirrors how “real” banking platforms are structured. It makes testing each layer in the system straight-forward.

SQLite was chosen because the application is locally hosted for demonstration purposes and does not handle any real data. SQLite is sufficient and requires no additional infrastructure setup.

Alternatives were not considered simply because this design is the simplest way to create and implement a model banking system in a short amount of time.

4. DATA DESIGN

4.1 Data Description

The information in this system is turned into a small set of related data structures that represent users, accounts, transactions, and system activity. Since the project is a simulated online banking system, all data is test data stored locally and handled by a mocked backend rather than any real banking network. The system needs to store login information, account details, balances, transaction records, and administrator activity so it can support the required customer and admin functions.

The main entity is the User. A user has login information such as username, password hash, and role. The role is important because the system uses role based access control, which means customers and administrators do not have access to the same features. A customer can view accounts and perform banking actions, while an administrator can manage test accounts and review logs.

Each customer is connected to one or more Accounts. An account stores an account identifier, account number, account type such as checking or savings, current balance, and the user who owns it. These account records are used to build the dashboard and account detail pages.

Each account is also connected to many Transactions. A transaction stores the transaction date, type, amount, the related account, and optional source or destination account information for transfers. Deposits, withdrawals, and transfers are all recorded as transactions so the system can show transaction history in chronological order and keep account balances consistent.

The system should also keep Activity Logs or Audit Logs. These records store actions such as login attempts, successful logins, failed logins, transfers, deposits, withdrawals, and admin

changes to test accounts. These logs support the administrator monitoring and log viewing requirements.

A simple local database can be used to organize this information. For example, the project can use tables such as Users, Accounts, Transactions, and ActivityLogs. If needed, a small configuration file can also store environment settings such as session timeout length and database connection values.

4.2 Data Dictionary

Below is a plain data dictionary in alphabetical order. Since this project is naturally described in an object oriented way, this section lists the main objects, their attributes, and their methods.

Account

Type: Object / database entity
Description: Stores one bank account belonging to a customer.

Attributes

accountId : int unique internal identifier for the account
accountNumber : string displayed account number
accountName : string display name for the account
accountType : string checking or savings
balance : decimal current account balance
ownerUserId : int user who owns the account

Methods

getSummary() returns account name, number, type, and balance
getDetails() returns full account information
deposit(amount) adds money to the account balance
withdraw(amount) removes money from the balance if funds are available
transferTo(destinationAccount, amount) moves money from this account to another account

ActivityLog

Type: Object / database entity
Description: Stores records of important system events for admin review.

Attributes

logId : int unique internal identifier for the log
userId : int user related to the action
actionType : string login, failed login, deposit, withdrawal, transfer, account update, and so on

actionDateTime : datetime when the event happened
details : string extra information about the event

Methods

recordEvent(userId, actionType, details) creates a new log record
getLogs() returns logs for admin review
filterLogs(criteria) returns logs based on search or filter conditions

Administrator

Type: Object / subclass of User
Description: Represents a user with admin privileges.

Attributes

Inherits all User attributes

Methods

createTestAccount(userInfo, accountInfo) creates a new customer and or account
modifyAccount(accountId, updatedInfo) changes test account information
viewSystemActivity() shows activity records
viewTransactionLogs() shows transaction log records

Customer

Type: Object / subclass of User
Description: Represents a normal banking user.

Attributes

Inherits all User attributes

Methods

viewDashboard() displays customer account summaries
viewTransactionHistory(accountId) displays account transaction history
makeTransfer(sourceAccountId, destinationAccountId, amount) transfers funds
makeDeposit(accountId, amount) performs a deposit
makeWithdrawal(accountId, amount) performs a withdrawal

Session

Type: Object / temporary data structure
Description: Stores login session information for the current user.

Attributes

sessionToken : string unique session identifier
userId : int logged in user
createdAt : datetime session start time
expiresAt : datetime session expiration time
isActive : boolean whether the session is still active

Methods

startSession(userId) creates a session after successful login
validateSession(token) checks whether the session is valid
endSession() logs the user out
isTimedOut() checks for timeout due to inactivity

Transaction

Type: Object / database entity
Description: Stores a deposit, withdrawal, or transfer record.

Attributes

transactionId : int unique internal identifier
accountId : int account tied to the transaction
transactionType : string deposit, withdrawal, transfer in, transfer out
amount : decimal amount of money involved
transactionDate : datetime date and time of transaction
sourceAccountId : int/null source account for transfers
destinationAccountId : int/null destination account for transfers
description : string optional text description

Methods

createTransaction(accountId, type, amount, sourceAccountId, destinationAccountId) saves a new transaction record
getTransactionHistory(accountId) returns all transactions for an account in chronological order

User

Type: Object / database entity
Description: Stores login and identity information for a system user.

Attributes

userId : int unique internal identifier
username : string login name
passwordHash : string securely stored password value
role : string customer or administrator
firstName : string user first name
lastName : string user last name
email : string contact email
lastLogin : datetime most recent login time

Methods

login(username, password) validates credentials
logout() ends the session
authenticate(password) checks password against stored hash
getRole() returns the user role

5. COMPONENT DESIGN

User Object
Function: login(username, password)
INPUT: username, password

userRecord = find user by username

IF userRecord does not exist
    record failed login attempt
    return "invalid login"
END IF

IF password does not match stored password hash
    record failed login attempt
    return "invalid login"
END IF

create new session for userRecord.userId
update user's last login time
record successful login
return "login success"

Local data:
userRecord stores the user found in the database
sessionToken stores the generated session identifier
Function: authenticate(password)
INPUT: password

compare input password to stored password hash

IF match is true
    return true
ELSE
    return false
END IF

Local data:
match stores the result of the password comparison
Function: logout()
find active session for current user
invalidate or remove session
record logout activity
return "logout success"

Function: getRole()
return role

Customer Object
Function: viewDashboard()
accounts = get all accounts that belong to current customer

FOR each account in accounts
    collect account name
    collect account number
    collect account type
    collect current balance
END FOR

return account summary list

Local data:

```
accounts stores all account objects for the customer
accountSummaryList stores the display data for the dashboard
Function: viewTransactionHistory(accountId)
account = find account by accountId

IF account does not belong to current customer
    return "access denied"
END IF

transactions = get all transactions for accountId
sort transactions by transaction date
return transactions

Local data:
account stores the selected account
transactions stores the transaction list
Function: makeTransfer(sourceAccountId, destinationAccountId, amount)
sourceAccount = find account by sourceAccountId
destinationAccount = find account by destinationAccountId

IF sourceAccount does not belong to current customer
    return "access denied"
END IF

IF destinationAccount does not belong to current customer
    return "access denied"
END IF

IF amount <= 0
    return "invalid amount"
END IF

IF sourceAccount.balance < amount
    return "insufficient funds"
END IF

sourceAccount.balance = sourceAccount.balance - amount
destinationAccount.balance = destinationAccount.balance + amount

create transfer-out transaction for sourceAccount
```

```
create transfer-in transaction for destinationAccount

save updated balances
save both transaction records
record transfer activity

return "transfer success"

Local data:
sourceAccount stores the account money is taken from
destinationAccount stores the account money is sent to
transferOutTransaction stores the source transaction record
transferInTransaction stores the destination transaction record
Function: makeDeposit(accountId, amount)
account = find account by accountId

IF account does not belong to current customer
   return "access denied"
END IF

IF amount <= 0
   return "invalid amount"
END IF

account.balance = account.balance + amount

create deposit transaction
save updated balance
save transaction
record deposit activity

return "deposit success"

Local data:
account stores the selected account
depositTransaction stores the transaction record
Function: makeWithdrawal(accountId, amount)
account = find account by accountId

IF account does not belong to current customer
```

```
    return "access denied"
END IF

IF amount <= 0
    return "invalid amount"
END IF

IF account.balance < amount
    return "insufficient funds"
END IF

account.balance = account.balance - amount

create withdrawal transaction
save updated balance
save transaction
record withdrawal activity

return "withdrawal success"

Local data:
account stores the selected account
withdrawalTransaction stores the transaction record

Administrator Object
Function: createTestAccount(userInfo, accountInfo)
IF current user's role is not administrator
    return "access denied"
END IF

create new customer user record using userInfo
create new account record using accountInfo
link account to new customer
save user record
save account record
record admin account creation activity

return "account created"

Local data:
```

```
userInfo stores the new customer's information
accountInfo stores the new account's information
newUser stores the created user object
newAccount stores the created account object
Function: modifyAccount(accountId, updatedInfo)
IF current user's role is not administrator
    return "access denied"
END IF

account = find account by accountId

IF account does not exist
    return "account not found"
END IF

apply updatedInfo to account
save account changes
record admin modification activity

return "account updated"

Local data:
account stores the target account
updatedInfo stores the new values to apply
Function: viewSystemActivity()
IF current user's role is not administrator
    return "access denied"
END IF

logs = get all activity log records
return logs

Local data:
logs stores the list of activity log records
Function: viewTransactionLogs()
IF current user's role is not administrator
    return "access denied"
END IF

transactions = get all transaction records
```

```
sort transactions by transaction date
return transactions

Local data:
transactions stores the full transaction log lis

Account Object
Function: getSummary()
summary.accountName = accountName
summary.accountNumber = accountNumber
summary.accountType = accountType
summary.balance = balance

return summary

Local data:
summary stores the account display information
Function: getDetails()
details.accountId = accountId
details.accountNumber = accountNumber
details.accountName = accountName
details.accountType = accountType
details.balance = balance
details.ownerUserId = ownerUserId

return details

Local data:
details stores the full account information
Function: deposit(amount)
IF amount <= 0
    return "invalid amount"
END IF

balance = balance + amount
return "deposit applied"

Function: withdraw(amount)
IF amount <= 0
    return "invalid amount"
```

```
END IF

IF balance < amount
   return "insufficient funds"
END IF

balance = balance - amount
return "withdrawal applied"

Function: transferTo(destinationAccount, amount)
IF amount <= 0
   return "invalid amount"
END IF

IF balance < amount
   return "insufficient funds"
END IF

balance = balance - amount
destinationAccount.balance = destinationAccount.balance + amount

return "transfer applied"

Local data:
destinationAccount stores the target account object

Transaction Object
Function: createTransaction(accountId, type, amount, sourceAccountId,
destinationAccountId)
newTransaction.accountId = accountId
newTransaction.transactionType = type
newTransaction.amount = amount
newTransaction.transactionDate = current date and time
newTransaction.sourceAccountId = sourceAccountId
newTransaction.destinationAccountId = destinationAccountId

save newTransaction
return newTransaction

Local data:
```

newTransaction stores the new transaction record before saving
Function: getTransactionHistory(accountId)
transactions = find all transactions for accountId
sort transactions by transaction date
return transactions

Local data:
transactions stores the list of matching transactions

ActivityLog Object
Function: recordEvent(userId, actionType, details)
newLog.userId = userId
newLog.actionType = actionType
newLog.details = details
newLog.actionDateTime = current date and time

save newLog
return newLog

Local data:
newLog stores the new activity record
Function: getLogs()
logs = retrieve all activity log records
sort logs by actionDateTime
return logs

Local data:
logs stores the full list of activity records
Function: filterLogs(criteria)
logs = retrieve all activity log records
filteredLogs = empty list

```
FOR each log in logs
   IF log matches criteria
      add log to filteredLogs
   END IF
END FOR
```

return filteredLogs

Local data:
logs stores all log records
filteredLogs stores only the records that match the filter
criteria may include user, action type, or date range

Session Object
Function: startSession(userId)
create unique session token
set createdAt to current date and time
set expiresAt based on timeout rule
set isActive to true
save session
return session token

Local data:
sessionToken stores the generated token
createdAt stores session start time
expiresAt stores session end time
Function: validateSession(token)
session = find session by token

IF session does not exist
    return false
END IF

IF session.isActive is false
    return false
END IF

IF current date and time > session.expiresAt
    session.isActive = false
    save session
    return false
END IF

return true

Local data:
session stores the located session record
Function: endSession()

```
set isActive to false
save session
return "session ended"

Function: isTimedOut()
IF current date and time > expiresAt
   return true
ELSE
   return false
END IF
```

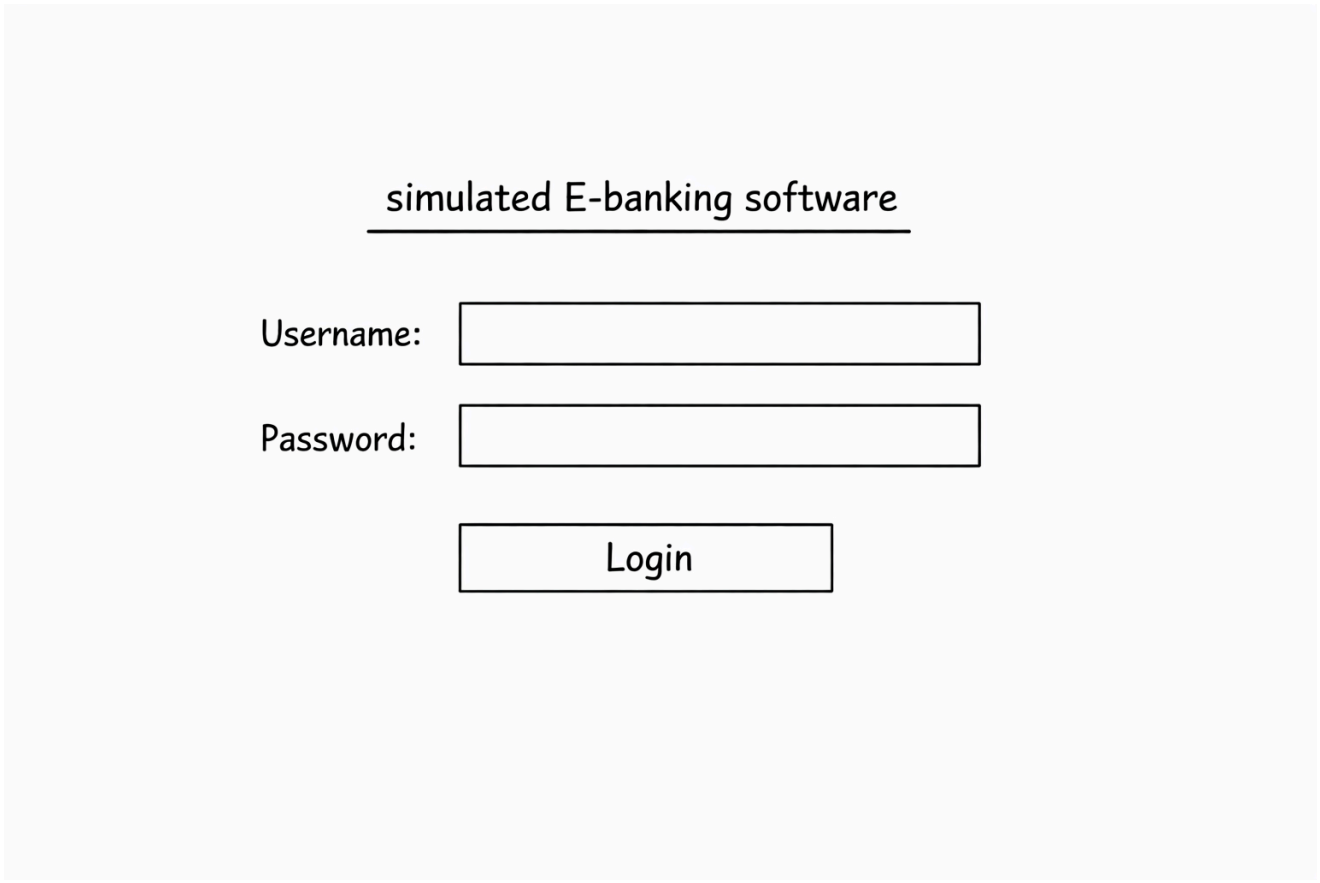
6. HUMAN INTERFACE DESIGN

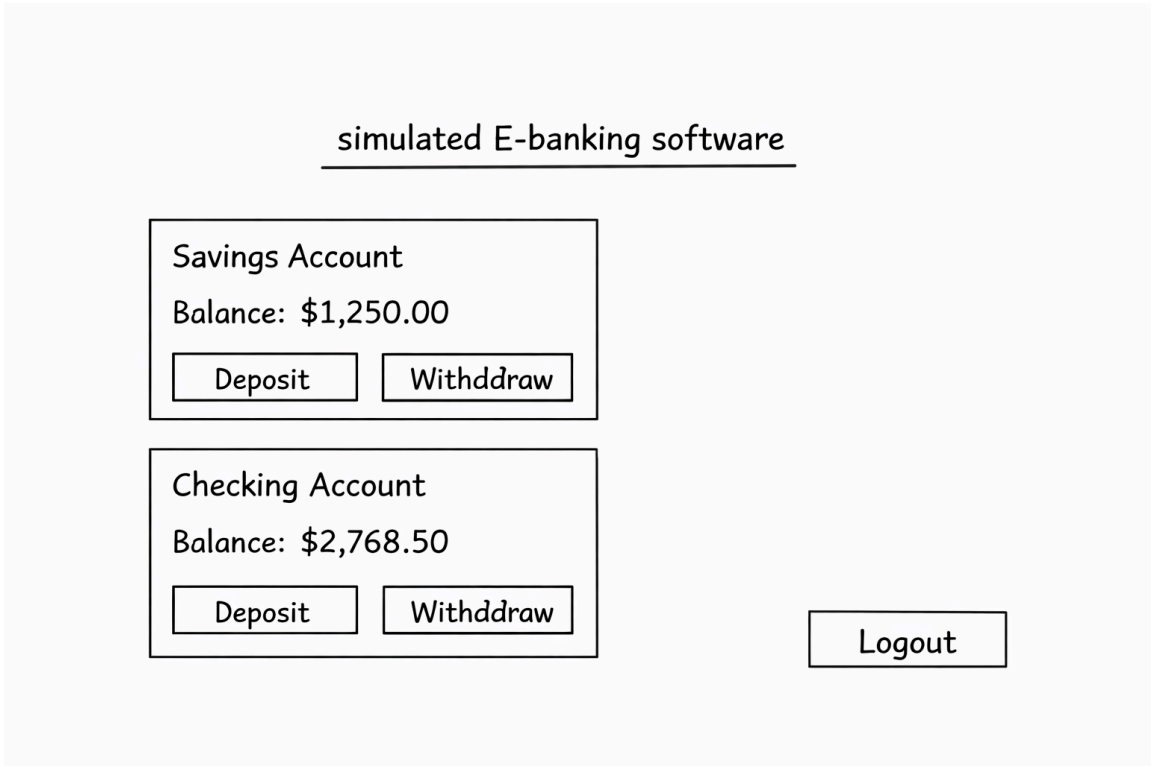
6.1 Overview of User Interface

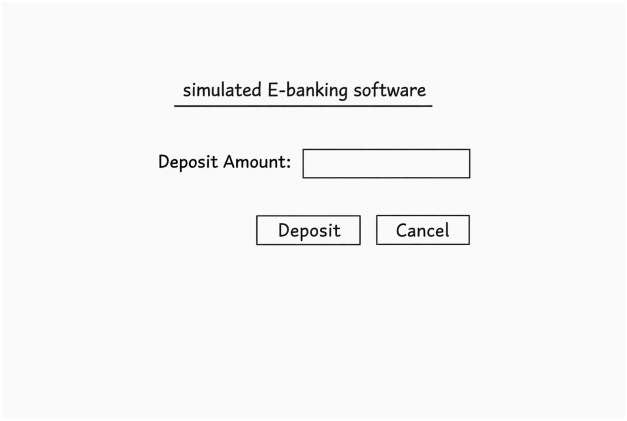
The user starts on a simple login screen where they enter their username and password and press the login button. If the login is successful, they are taken to the main account page. On this page, they can see their checking and savings accounts along with the current balances. Each account has buttons to either deposit or withdraw money. The user can choose one of these actions, which takes them to a simple page where they enter an amount and confirm or cancel the action. After completing a deposit or withdrawal, the system updates the balance and records the transaction, then returns the user to the account page.

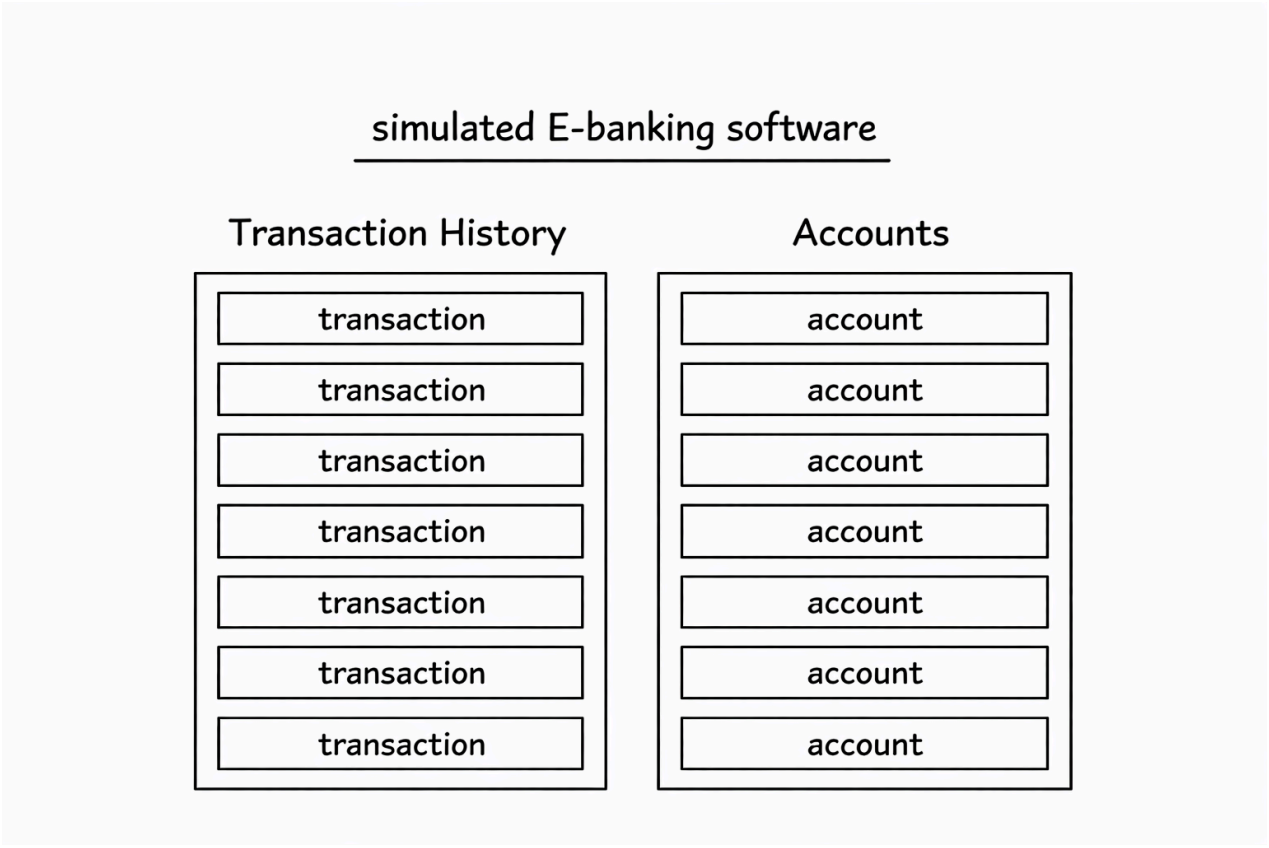
The user can also log out at any time using the logout button, which ends their session and returns them to the login screen. If the user is an administrator, they can access a different view where they see lists of accounts and transactions. These are shown as simple lists that allow the admin to review system activity and manage test accounts. Overall, the system is navigated through clear buttons and simple pages, with each action leading directly to the next step and then returning to the main account view.

6.2 Screen Images


simulated E-banking software
Username:
Password:
Login


simulated E-banking software
Savings Account
Balance: $1,250.00
Deposit
Withddraw
Checking Account
Balance: $2,768.50
Deposit
Withddraw
Logout





6.3 Screen Objects and Actions

The system includes several specific input boxes and buttons, each tied directly to a system function. On the login screen, the username text box allows the user to enter their username, and the password text box allows them to enter their password securely. The login button submits these credentials to the system, which then checks them against stored user data

to either grant or deny access. On the deposit page, the deposit amount text box allows the user to enter how much money they want to add to the selected account. The deposit button confirms the action, updates the account balance, and creates a transaction record, while the cancel button stops the operation and returns the user to the previous screen without making changes. On the withdrawal page, the withdraw amount text box allows the user to enter how much money they want to remove. The withdraw button attempts to subtract that amount from the account if there are enough funds, updates the balance, and records the transaction, while the cancel button again exits without making any changes.

On the main account screen, each account has its own set of buttons. The deposit button for an account opens the deposit page for that specific account, and the withdraw button for an account opens the withdrawal page for that account. These buttons ensure that the action is tied to the correct account. The logout button ends the user's session, clears their session data, and returns them to the login screen. In the administrator view, the screen displays labeled sections for transaction history and accounts, where each box labeled "transaction" represents a recorded transaction and each box labeled "account" represents a user account. While these boxes are mainly for viewing, they represent the underlying data that administrators can monitor, such as account details and transaction activity within the system.

7. REQUIREMENTS MATRIX

Requirement ID	Design component to satisfy	Test case(s)	Status
UI-REQ-001	6.2 Screen images 6.3 Screen objects and actions		Incomplete
UI-REQ-002	6.2 Screen images 6.3 Screen objects and actions		Incomplete
UI-REQ-003	6.2 Screen images 6.3 Screen objects and actions		Incomplete
UI-REQ-004	6.2 Screen images 6.3 Screen objects and actions		Incomplete
UI-REQ-005	6.2 Screen images 6.3 Screen objects and actions		Incomplete
HARDWARE-REQ-001			
HARDWARE-REQ-002			
SOFTWARE-REQ-001			
COMMS-REQ-001			
COMMS-REQ-002			
LOGIN-REQ-001	6.2 Screen images 6.3 Screen objects and actions		Incomplete
LOGIN-REQ-002	User object		Incomplete
LOGIN-REQ-003	User object		Incomplete
LOGIN-REQ-004	User object		Incomplete
LOGIN-REQ-005	User object		Incomplete
ACCOUNT-REQ-001	Customer object		Incomplete

ACCOUNT-REQ-002	Customer object		Incomplete
ACCOUNT-REQ-003	Customer object		Incomplete
ACCOUNT-REQ-004	Customer object		Incomplete
TRANSACTION-REQ-001	Customer object Transaction object		Incomplete
TRANSACTION-REQ-002	Customer object Transaction object		Incomplete
TRANSACTION-REQ-003	Transaction object		Incomplete
TRANSFER-REQ-001	Customer object		Incomplete
TRANSFER-REQ-002	Customer object		Incomplete
TRANSFER-REQ-003	Customer object		Incomplete
TRANSFER-REQ-004	Customer object		Incomplete
TRANSFER-REQ-005	Customer object		Incomplete
TRANSFER-REQ-006	Customer object		Incomplete
TRANSFER-REQ-007	Customer object		Incomplete
TRANSACTION-REQ-004	Customer object		Incomplete
TRANSACTION-REQ-005	Customer object		Incomplete
TRANSACTION-REQ-006	Customer object		Incomplete
TRANSACTION-REQ-007	Customer object		Incomplete
TRANSACTION-REQ-008	Customer object		Incomplete
TRANSACTION-REQ-009	Customer object		Incomplete
ADMIN-REQ-001	Admin object		Incomplete
ADMIN-REQ-002	Admin object		Incomplete
ADMIN-REQ-003	Admin object		Incomplete

ADMIN-REQ-004	Admin object		Incomplete
ADMIN-REQ-005	Admin object		Incomplete
ADMIN-REQ-006	Admin object		Incomplete
ADMIN-REQ-007	Admin object		Incomplete
ADMIN-REQ-008	Admin object		Incomplete
ADMIN-REQ-009	Admin object		Incomplete
PERF-REQ-001			Incomplete
PERF-REQ-002			Incomplete
PERF-REQ-003			Incomplete
DB-REQ-001	Customer object		Incomplete
DB-REQ-002	Customer object Account object		Incomplete
DB-REQ-003	Customer object Account object		Incomplete
DB-REQ-004	User object		Incomplete
DESIGN-REQ-001			
DESIGN-REQ-002			
DESIGN-REQ-003			
STD-REQ-001			
STD-REQ-002			
STD-REQ-003			
REL-REQ-001	Customer object Account object Transaction object		Incomplete
REL-REQ-002	Customer object		Incomplete
AVAIL-REQ-001			Incomplete
AVAIL-REQ-002			Incomplete
SEC-REQ-001	User object		Incomplete

SEC-REQ-002	User object Admin object		Incomplete
SEC-REQ-003	User object		Incomplete
MAINT-REQ-001			
MAINT-REQ-002			
MAINT-REQ-003			